

Program Learning with Verifiable Rewards: Symbolic Backpropagation for Post-Training LLMs

Vishvesh Bhat

CoreThink AI

vish@corethink.ai

August 31, 2026

Abstract

Post-training a language model to reason means updating its weights. Supervised fine-tuning and reinforcement learning both place the acquired capability inside the model, where it cannot be inspected, cannot be checked step by step, and cannot be moved to another model. We argue that for tasks whose intermediate steps admit verification, reasoning is better placed *outside* the base model’s weights, as an explicit program composed from deterministic and neural primitives. We introduce PLVR (Program Learning with Verifiable Rewards), a post-training method that learns such programs directly from input–output examples. Its mechanism is *symbolic backpropagation*: each program layer carries a typed ontology, a loss is computed at the output against ground truth, and required input ontologies are propagated backward by type inference over primitive signatures—an analogue of the chain rule in which credit assignment is a derivation rather than an estimate. Where RLVR verifies a terminal outcome, PLVR’s reward is a per-step contract verdict, dense over program structure. On LiveCodeBench v6 and τ^2 -Bench, $\approx 30\text{B}$ base models with PLVR outperform RL at matched budget by 27.8 points on average, and frontier models an order of magnitude larger by 13.6 points. A single primitive library serves two benchmarks, so the marginal cost of a new task is ≈ 100 examples of program search and no new fine-tuning data; identification of the best available program is reliable within seventy. Replacing the loss-guided search with uniform sampling over the same type-admissible space, at equal budget, collapses the median program from 65.6% to 17.5%, identifying the backward pass rather than the type system as the source of the advantage. We release the symbolic backpropagation library and a conformance checker so the method can be applied to primitive libraries other than our own.

1 Introduction

Post-training has converged on a single locus. Supervised fine-tuning adjusts weights toward demonstrated behaviour; reinforcement learning adjusts them toward rewarded outcomes (Guo et al., 2025). Both store what was learned in the same place it was already stored, and inherit three properties of that choice. The acquired capability is opaque: nothing in the weights says which step of a multi-step task the model now performs better. It is unverifiable in progress: a trajectory can only be judged once complete. And it is bound to one model: capability learned for a 12B checkpoint does not transfer to a 30B one without repeating the procedure.

For a large class of tasks, none of this is necessary. Agentic tool-calling, policy-constrained dialogue, and program generation all decompose into steps whose intermediate outputs can be checked—a tool call either matches a signature or does not, a derived task list either covers the request or does not, an ordering either respects declared dependencies or does not. Where such checks exist, the reasoning that composes those steps can live in an explicit artifact rather than in weights.

Reasoning outside the base model. We take this position concretely. A *reasoning program* is a composition of primitives—some deterministic, some small fine-tuned language models—arranged by a typed domain-specific language, where every primitive carries an output contract enforced at execution. The base model is not updated. What is learned is the program: which primitives, in what arrangement, with what parameters.

This raises the question the rest of the paper answers. Gradient methods learn because backpropagation assigns credit through a differentiable composition. A program of discrete, partly symbolic primitives is not differentiable,

and the obvious substitute—search over program structure guided by a scalar score, as in recent agentic workflow optimisation (Hu et al., 2025; Zhang et al., 2025)—estimates credit from coarse signals over whole trajectories. How should credit be assigned through a program that cannot be differentiated?

Symbolic backpropagation. Our answer is that in a typed, contract-verified program, credit need not be estimated at all. A program is organised into N layers, each carrying an ontology: a structured, typed representation of the state at that point, and an inspection point in the sense of a debugger breakpoint. A forward pass produces an output ontology, compared against ground truth by a loss that treats each object as a separate axis and orders type errors strictly before value errors.

The backward pass then propagates *required ontologies* rather than gradients. Given the ontology required at layer k and the declared signatures of the primitives, the compositions that could produce it induce, by type inference, the ontology required at layer $k - 1$. That derived requirement plays the role the upstream gradient plays in the differentiable case. Type resolution is deterministic; only the search over parameter values within each primitive’s declared range is not. Contract verdicts prune this search and localise failure to the step that produced it.

Verifiable rewards, densified. The relationship to RLVR is direct. RLVR replaced learned reward models with programmatically verifiable outcomes—a signal checked rather than modelled. That signal remains terminal and scalar: it reports that a trajectory succeeded or failed, not where it went wrong. PLVR extends verifiability inward. Every step carries a typed output contract, so correctness is checkable at each step. The reward is verifiable in RLVR’s sense—no learned model, checked against a specification—but dense over the program’s structure rather than sparse over its trajectory.

Scope. The method applies where intermediate outputs admit checking. This is a precondition entailed by the mechanism, not a limitation discovered after the fact: contracts that localise failure require checkable outputs. Tasks whose quality is a matter of judgement rather than satisfaction—summarisation, open generation—remain the domain of gradient post-training, and we make no claim about them. We compare only against reinforcement-learning post-training, which is the stronger target, and make no claim relative to supervised fine-tuning.

Contributions.

- **Symbolic backpropagation**, a credit-assignment procedure for compositional neural programs in which required input ontologies are derived backward by type inference over primitive signatures rather than estimated from rollouts. Types resolve deterministically; value search is bounded by declared ranges.
- **PLVR**, a post-training method built on it, alternating program search under frozen primitives with primitive retraining under a fixed program. The base model’s weights are never updated.
- **Evidence that program learning outperforms RL post-training at matched budget** on two benchmarks with distinct verification characters—LiveCodeBench (Jain et al., 2024) (execution against tests) and τ^2 -Bench (Barres et al., 2025) (constraint satisfaction over dialogue), the latter across three domains—against vanilla prompting, a token-matched agentic harness (Zhang et al., 2026), and published RL checkpoints matched to us on training data and on parameter count respectively.
- **An ablation isolating the mechanism.** Replacing the loss-guided search with uniform sampling over the identical type-admissible space, at equal budget and with every other constraint retained, collapses the median program from 65.6% to 17.5%. The advantage is attributable to the backward pass rather than to the type system, which both arms share.
- **Amortisation.** One primitive library of 19,627 synthetic examples serves both benchmarks. Where RL post-training requires a task-specific corpus per task, the marginal cost of a new task here is ≈ 100 examples of program search and no new fine-tuning data; identification of the best available program is reliable within seventy.
- **A released library and conformance checker**, so that symbolic backpropagation can be applied to any primitive set satisfying the interface, not only ours.

What a reader can do with this. The result we consider most portable is not a benchmark number. It is a conjecture about substrates: that running a program-construction procedure over typed, contract-verified primitives outperforms running the same procedure over raw model calls. Our ablation establishes the weaker version—that a loss-guided derivation over such a substrate beats uniform sampling over it—but the general claim concerns any construction procedure, not only ours, and we did not test it with a third-party method. It is checkable by anyone with a compositional pipeline and a way to verify its intermediate steps, and it does not require our code.

2 Related Work

2.1 Post-training and verifiable rewards

Supervised fine-tuning and reinforcement learning are the two established routes to improving a model’s reasoning. RLHF trains against learned reward models fitted to human preferences (Christiano et al., 2023; Ouyang et al., 2022). Reinforcement learning with verifiable rewards (RLVR) replaces the learned reward model with a programmatic check—answer matching for mathematics, execution for code—removing a component that could be gamed and a source of reward hacking. DeepSeek-R1 develops extended chain-of-thought, reflection, and self-verification through large-scale RL over such rewards (Guo et al., 2025).

We share RLVR’s premise that a reward one can check is preferable to one that must be modelled, and extend it in a single respect: the verifiable signal in RLVR is terminal and scalar, reporting that a trajectory succeeded or failed rather than where it went wrong. In PLVR the signal is a typed output contract at every step, verifiable in the same sense and dense over the program’s structure. The base model’s weights are not updated at all; what is learned is an explicit program together with the weights of the small primitives it invokes.

2.2 Process supervision and step-level verification

The limitation of terminal rewards is well recognised. Process supervision provides step-level rather than outcome-level feedback, and process reward models (PRMs) trained on step-wise labels improve reliability over outcome reward models (Uesato et al., 2022; Lightman et al., 2023). Subsequent work reduces the annotation burden through automated label construction (Wang et al., 2024) and extends PRMs to agentic settings, where—unlike mathematical reasoning—the correctness of an individual action is not clear-cut and must be estimated from eventual outcomes (Xi et al., 2025). A recent line trains step-level verifiers using formal verification tools rather than human annotation (Kamoi et al., 2026).

This is the nearest work to ours on the axis of signal density, and the distinction is precise. A PRM is a *learned* verifier: a model that judges steps, requiring step-level supervision to train and inheriting the same gameability and opacity that RLVR removed at the outcome level. A contract in PLVR is a *programmatic* check derived from a primitive’s declared type signature. It requires no step-level annotation, it is checked rather than predicted, and where the check is structural it is exhaustive. In short, PRMs extend reward *modelling* to the step level; we extend reward *verification* to the step level.

2.3 Backpropagation through compound AI systems

A substantial line of work carries the backpropagation metaphor into systems built from LLM calls. ProTeGi introduced natural-language gradients for prompt search (Pryzant et al., 2023). TextGrad formalises this into an end-to-end framework in which a computation graph over text variables is optimised by backpropagating LLM-generated critiques (Yuksekgonul et al., 2024). Trace generalises the idea to execution traces under the OPTO formalism (Cheng et al., 2024). GEPA shows that reflective prompt evolution using natural-language feedback can outperform reinforcement learning while requiring far fewer rollouts (Agrawal et al., 2026).

Our method shares the metaphor and differs in what the backward signal *is*. In every system above, the gradient analogue is text produced by a language model: a critique, a suggestion, a reflection. It is therefore stochastic, unverifiable, and subject to the degradation with depth that motivates the analyses cited. In PLVR the backward signal is a *required ontology* derived by type inference over declared primitive signatures, together with contract verdicts that localise failure to the step that produced it. Nothing in the backward pass is generated; the required input follows from the signatures. This is the sense in which credit assignment here is a derivation rather than an estimate, and it is why the analogue does not degrade with depth in the way textual feedback does.

We note that GEPA’s thesis—that language provides a richer learning medium than policy gradients derived from sparse scalar rewards—is close to our own diagnosis of why gradient post-training is weak on multi-step tasks. We differ in the remedy: GEPA searches over instructions guided by reflective feedback, while we obtain per-step attribution structurally, from types and contracts.

2.4 Declarative pipelines and workflow synthesis

DSPy abstracts LM pipelines as parameterised modules compiled by an optimiser over prompts and demonstrations (Khattab et al., 2023). A more recent line searches over pipeline *structure* rather than pipeline text. ADAS treats an agent as a program and uses a meta-agent to propose, execute, and retain high-scoring designs (Hu et al., 2025). AFlow reformulates the problem as Monte Carlo tree search over code-represented workflows in which LLM-invoking nodes are connected by edges, refined through code modification and execution feedback (Zhang et al., 2025). AgentFlow uses reinforcement learning to construct workflows that dynamically select tools from a predefined set (Li et al., 2026).

These share our interest in explicit, searchable program structure. They differ in two respects. First, their optimisation signal is a coarse scalar score over whole trajectories; ours is a per-object loss with per-step contract attribution. Second, they optimise structure over untyped LLM calls without verification of intermediate semantic outputs, whereas our primitives carry typed contracts enforced at execution.

2.5 Neurosymbolic reasoning with language models

Logic-LM and Proof-of-Thought delegate reasoning to symbolic solvers or theorem provers, in domains with clean formal semantics (Pan et al., 2023; Ganguly et al., 2024). SEVerA pairs first-order-logic contracts with verified fallback mechanisms (Banerjee et al., 2026).

Agentic tool-calling does not. Intent coverage and function matching are semantic judgements that resist complete formal specification. PLVR therefore splits the problem rather than choosing a side: deterministic mechanisms enforce structural invariants through typed composition and execution guards, while learned primitives handle the semantic decisions, bounded by contracts that are empirically validated rather than proven. Our use of “verification” throughout denotes contract checking plus empirical validation, not end-to-end formal correctness, and we state this explicitly rather than allowing the term to carry more weight than the method supports.

3 Methodology

A PLVR program is a composition of primitives arranged by a typed DSL, organised into N layers. Each layer carries an *ontology*: a structured, typed representation of the state at that point in the computation. Learning a program means determining, for each layer, which primitives and operators produce the ontology the next layer requires.

We describe the representation (§3.1–3.4), the loss over ontologies (§3.5), the backward pass that propagates required ontologies by type inference (§3.6), and the alternation between program search and primitive training (§3.7).

3.1 Ontologies and layers

An *object* is a typed record. An *ontology* is a keyed collection of objects:

$$\text{Obj} = \{ \text{type} : T, \text{values} : K \rightarrow (V \mid \text{Obj} \mid [\text{Obj}]) \} \quad (1)$$

$$\Omega = K \rightarrow \text{Obj} \quad (2)$$

Objects nest, so an ontology carries both type structure and instance values for the example being processed. We use the term in the knowledge-representation sense—classes and relations—with the qualification that a layer’s ontology is *populated*: the same object carries its type and the values realised on this input.

Ontologies enter and leave the program through a pair of tokenizers, described in §3.2.

A program is a sequence of N layers. Every layer is initialised as pass-through, and each is an inspection point: at any layer the ontology can be read directly, giving the intermediate state of the computation in the same sense a debugger breakpoint gives the state of a running program. This is a property of the representation rather than an added instrumentation—the ontology *is* the state.

3.2 The neurosymbolic tokenizer

A program operates over ontologies, but benchmarks supply raw text and expect raw answers. A single tokenizer bounds the program at both ends:

$$\text{TOKENIZE} : \text{Raw} \times \text{Schema} \rightarrow \Omega \quad (3)$$

applied to the instance to produce Ω_0 and to an answer to produce Ω_{out} . Both are ontologies in exactly the same format—keyed collections of typed, nestable objects. What differs is only which object types the schema declares, and therefore which are detected: an instance yields user turns, assistant turns, tool descriptors, and configuration; an answer yields code and tool calls.

This uniformity is what makes the layer stack coherent. Every layer, including the first and the last, carries an ontology of the same kind, so the loss (§3.5) applies unchanged at the output boundary and at the input boundary (§3.6), and a primitive that consumes a type does not care whether that object arrived from the instance, from an intermediate layer, or from an answer being scored.

The tokenizer plays the role a subword tokenizer plays for a transformer, converting between the external representation and the units the system computes over. It is hybrid: a learned component identifies structure in unconstrained text, and symbolic components check that the structure found is well-formed and grounded. Hence *neurosymbolic*.

Contract. Largely deterministic, and identical in kind at both ends. Every object must conform to a declared type in the schema. Where a schema declares an ordering, indices must be contiguous and strictly increasing—an ontology is a keyed collection, so sequence survives as data rather than as position. Cross-references must resolve: a tool call in an assistant turn must name a tool present in the registry, and a call’s bindings must type-check against the signature of the function named. Where a declared type carries a formal language, its values must parse and type-check in it.

The constraint that matters most is that every text field must be a verbatim span of its source. Nothing may be paraphrased or invented at the boundary. Only semantic attribution—which speaker a turn belongs to, which action an answer expresses—is validated empirically.

Why the boundary matters. The tokenizer determines what the loss can see. An error it makes on the instance is invisible to every downstream contract, because the ontology it produced is what the program takes as ground truth about its input. This is why the verbatim-span constraint is deterministic rather than empirical: it is the one place a learned component could silently introduce content that nothing downstream would catch. At the other end, the tokenizer fixes the granularity of the loss—objects it does not distinguish cannot be scored separately, and objects it fabricates are penalised as spurious.

3.3 Primitives

A primitive is a leaf that computes. Each is narrowly scoped, pure (stateless and side-effect-free), carries a declared input/output type signature, and carries an *output contract*: an explicit specification its result must satisfy.

Primitives are symbolic or neural. Symbolic primitives have semantics expressible as a formal program and are correct by construction with respect to their specification. Neural primitives are small language models fine-tuned on a single operation; restricting each to one operation follows the empirical property that reliability is inversely related to task scope.

A primitive is *first-order*: it consumes and produces ontologies and does not take a composition as an argument. This is the rule that separates primitives from operators (§3.4), and it is why a primitive can carry an output contract at all—it has a result of its own to check.

The library used throughout this work contains 10 primitives: DECOMPOSE, ELABORATE, INIT-SCRATCHPAD, GET-ORDER, DENOTE, CHECK-PREREQUISITES, MATCH, IDENTITY, WRAP, and APPEND. We group them by role rather than by capability, which is what makes the set principled rather than fitted to a benchmark:

- **Lifting.** DENOTE maps text to a formal representation in a fixed target language; ELABORATE expands terms using a fixed dictionary. These bring unconstrained input into the regime where contracts apply.
- **State.** INIT-SCRATCHPAD seeds a monotone store from configuration; CHECK-PREREQUISITES evaluates a three-valued condition against it.
- **Structure.** DECOMPOSE derives an ordered task list; GET-ORDER produces a dependency-respecting permutation.
- **Binding.** MATCH selects a function and resolves its parameters by prioritised lookup.
- **Structural.** IDENTITY passes an ontology through unchanged; WRAP lifts a value into a singleton collection ($\alpha \rightarrow [\alpha]$); APPEND concatenates values into a collection. All three are fully deterministic and first-order.

Primitive	Learned component	Symbolic component
IDENTITY	—	pass-through
WRAP	—	singleton lift
APPEND	—	concatenation
INIT-SCRATCHPAD	—	store construction
ELABORATE	expansion	dictionary lookup, splice
DENOTE	denotation	parse, type-check
GET-ORDER	dependency graph	cycle detect, topo-sort
CHECK-PREREQ.	semantic SAT	membership, branching
DECOMPOSE	task derivation	schema parse, registration
MATCH	function selection	lookup, execution guard

Four primitives are fully deterministic; the remaining six are hybrid, pairing a learned component for a semantic judgement with symbolic components that are correct by construction.

ELABORATE illustrates why the hybrid arrangement matters. Its output is free text, which admits no direct check. But because expansions are drawn from a fixed dictionary, its value constraints—that the input is a subsequence of the output, and that every inserted span is a dictionary entry—are checked exhaustively by symbolic components even though the expansion itself is learned. The learned part decides *which* terms to expand and where; the symbolic part guarantees that nothing was invented, dropped, or reordered. An unverifiable output kind is thereby brought under a deterministic contract without constraining the model that produces it.

Contracts span type constraints (a valid object conforming to a schema), value constraints (extracted entities must appear in the source), and semantic constraints (a decomposition must cover the original query). We distinguish two levels of assurance. *Deterministic* contracts are checked exhaustively and guarantee the specified property. *Empirical* contracts—those on neural primitives—detect outputs violating an expressible condition but do not prove that every contract-satisfying output is correct. Our use of “verification” throughout denotes contract checking plus empirical validation, not formal correctness.

3.4 Operators

Operators are *higher-order*: each takes a primitive or sub-composition as an argument and arranges it. Producing no result of their own, they carry typing rules and structural obligations rather than output contracts, enforced by the DSL’s typed composition at construction time.

Arity is the whole of the distinction. IDENTITY, WRAP, and APPEND are first-order and therefore primitives, despite being purely structural; everything below abstracts over a composition and therefore is not. WRAP in particular does substantial work during type resolution: a required $[\alpha]$ is frequently reachable from an available α by a single lift, and without it that gap would have to be bridged by a composition.

Operator	Signature
SEQUENCE	$(\alpha \rightarrow \beta) \times (\beta \rightarrow \gamma) \rightarrow (\alpha \rightarrow \gamma)$
PARALLEL	$(\alpha \rightarrow \beta) \times (\alpha \rightarrow \gamma) \rightarrow (\alpha \rightarrow \beta \times \gamma)$
BRANCH	$\text{Cond} \times (\alpha \rightarrow \beta) \times \alpha \rightarrow \alpha \mid \beta$
FOREACH	$(\alpha \rightarrow \beta) \times [\alpha] \rightarrow [\beta]$
LOOP	$(\alpha \rightarrow \alpha) \times \alpha \rightarrow \alpha$
FALLBACK	$(\alpha \rightarrow \beta) \times (\alpha \rightarrow \beta) \times \alpha \rightarrow \beta$
WRAP	$\alpha \rightarrow [\alpha]$

Control flow lives here, which is what distinguishes a program from a linear pipeline. Three properties matter for the backward pass.

BRANCH and LOOP are parametrically polymorphic: their output type is a variable that unifies with any required type, and instantiating it constrains the required input. The same holds for the IDENTITY primitive. All other primitives are monomorphic and admissible only on exact type match.

LOOP requires a type-preserving body, so cardinality varies across iterations but the type does not. FOREACH is not type-preserving but has determined cardinality ($|out| = |in|$) and structural termination, making its backward derivation exact and its contract totally checkable. Where a required output is a collection and a body maps its element type, FOREACH is preferred.

WRAP lifts a value into a singleton list. It is the adapter that makes FOREACH and APPEND composable with primitives producing a single object, and its backward derivation is exact: a required output $[\alpha]$ of length one implies a required input α . Together with APPEND it gives the search a route to collection types without requiring a primitive that produces them directly.

The IDENTITY primitive is the unit for SEQUENCE and the initialisation state of every layer. Because N is fixed, some tasks require fewer than N real steps; IDENTITY makes “this layer does nothing” a legitimate synthesis result rather than something the step must manufacture around. The count of non-IDENTITY layers in a converged program is its *effective depth*.

3.5 Ontology loss

The loss $L(\Omega_{\text{pred}}, \Omega_{\text{tgt}})$ compares a produced ontology against a target. It is keyed by $\text{dom}(\Omega_{\text{tgt}})$, so only requested outputs are scored. This is what allows one program to serve benchmarks with different output schemas: an unrequested Code object on a tool-calling example costs nothing, because the target does not ask for one.

Alignment. Predictions are matched to targets by *content*, not by key. Key-based alignment compares $\text{pred}[k]$ only ever to $\text{tgt}[k]$, which is wrong whenever a program is correct but offset. We observed this directly: a prediction containing every correct call, displaced by one setup call, received the maximum loss on both axes under key alignment despite differing from the target by a single insertion. Content alignment matches objects by what they contain and scores the residual, so an offset costs what an offset should cost.

Per-object loss. For each key $k \in \text{dom}(\Omega_{\text{tgt}})$,

$$\ell(k) = \begin{cases} (1, 1) & \text{object missing, or type mismatch} \\ (0, 1 - a(p_k, t_k)) & \text{otherwise} \end{cases} \quad (4)$$

where a is the agreement measure for the object’s type. Value comparison is not attempted across types: values of different types are not comparable, and partial credit across them would be spurious.

The agreement measure is declared per type rather than fixed globally, because what constitutes agreement differs by what the object is. In the configuration used throughout this work:

Type	Policy	$a(p, t)$
Code	TESTS	test pass rate
ToolCall	PARTIAL	exact match with partial credit

Exact match is the wrong measure for code: two programs may differ textually and behave identically, and a program may be nearly correct while matching nothing. Pass rate measures what is actually being asked. For tool calls, by contrast, exact match is meaningful—a call either names the right function with the right bindings or it does not—but graded partial credit over bindings is retained so that a call with the correct function and one wrong parameter is distinguished from a call to the wrong function entirely. Without that gradation the search receives no signal within a failing call.

Score. A loss map reduces to a comparable pair:

$$\mathcal{SC}(\ell) = \left(\underbrace{\sum_k \ell(k)_{\text{type}}}_{\text{integer}}, \underbrace{\sqrt{\sum_k \ell(k)_{\text{val}}^2}}_{\text{unbounded above 1}} \right) \quad (5)$$

compared lexicographically.

Two choices here are deliberate. The comparison is lexicographic rather than a weighted scalar: a scalar penalty assigning type errors a cost $1 + \lambda$ gives type dominance per object but not in aggregate, and restoring aggregate dominance requires $\lambda \geq |\text{dom}(\Omega_{\text{tgt}})|$, so λ would have to scale with ontology size. And the value axis uses an L_2 magnitude while the type axis uses a count: type errors are binary, so any L_p magnitude is a monotone function of the count and changes no ordering, whereas value errors are continuous and L_2 distinguishes one object sharing nothing with its target from several that are each partly correct.

We report the consequence rather than only the motivation. Lexicographic comparison is brittle by construction, and measurably so: one additional type error outranks any value advantage, however large. A candidate that is substantially closer on every value axis loses to one that is marginally better on types. This is the intended behaviour—types must be right before values are worth searching—but it may make the search sensitive to type errors arising from a single misclassified object.

Accumulation across examples. The beam ranks on cumulative score: a running count of type errors and a running sum of squared value errors, with the magnitude taken at comparison time. Per-example magnitudes are not summed, since that differs from the norm over all objects seen. Scores are not normalised by ontology size, so an example with more objects contributes more.

Ranking key. The loss is not the ranking key. Candidates are ordered on

$$(\text{UNMET}, \text{type_count}, \text{val}_{L_2}, \text{deviation}) \quad (6)$$

with size and label breaking any remaining ties. UNMET is set when a program fails to execute or returns no result, placing non-running candidates below every running one regardless of their partial loss. DEVIATION is a trust-region term: the symmetric difference between a candidate’s required input ontology and the incumbent’s at the same depth, $|\Omega_{\text{in}}(\text{cand}) \Delta \Omega_{\text{in}}(\text{inc})|$, refreshed from the previous example’s winner. Among candidates that are otherwise tied, it prefers those that stay near the current best program rather than re-deriving a distant one, which stabilises the beam across the sequential pass.

Loss and ranking are kept distinct on purpose. The loss is a statement about how wrong an output is; the ranking key adds search-control terms that are properties of a candidate’s relationship to the search state rather than of its output.

3.6 Symbolic backpropagation

A forward pass executes the program and records each layer’s ontology, producing a loss at the output. The backward pass propagates *required ontologies*.

Given the ontology Ω_{req} required at layer i , the *synthesis step* produces a ranked set of compositions that could yield it, each paired with the input ontology that composition would require at layer $i - 1$. That derived requirement becomes Ω_{req} for the next synthesis step, playing the role the upstream gradient plays in the differentiable case.

The step resolves types before values.

Type resolution. For each object $o \in \Omega_{\text{req}}$, first-order unification over primitive signatures identifies every primitive whose declared output type unifies with $\text{type}(o)$; monomorphic primitives require exact match, polymorphic operators unify with any type and propagate the instantiation to their required input. Where no single primitive covers Ω_{req} , operators compose candidates up to a depth bound d_{max} . For each surviving composition, the leaf signatures induce

$$\Omega_{\text{in}} = \bigcup \{ \text{intype}(p) \mid p \in \text{leaves}(c) \} \quad (7)$$

Compositions whose Ω_{in} requires an object unreachable from what is available below are discarded before execution.

Value resolution. Free parameters are enumerated across the composition and restricted to their declared ranges, further narrowed by values available in Ω_{in} , the scratchpad, and the turn context. The resulting space is searched for the assignment minimising value loss. Contract verdicts prune here: an assignment that would violate a primitive’s own output contract on this input is rejected without executing the remainder of the layer.

This is the sense in which credit assignment is a derivation rather than an estimate. Type resolution is deterministic—the required input follows from the signatures—and only the search over values within declared ranges is not.

Scoring and truncation. Candidates are executed on the example through the already-synthesised layers and scored against ground truth, not against Ω_{req} alone: a composition that hits its local target but leads to a worse end result must rank below one that approximates it and ends better. Compositions behaviourally equivalent to IDENTITY on the trace are canonicalised and deduplicated. The beam retains the top k programs globally—across all programs seen, not per layer—which bounds growth that would otherwise be exponential in depth.

Termination at layer 1. At every layer above the first, the derived Ω_{in} is free: it becomes the next step’s target. At layer 1 there is no next step, and the composition must be satisfiable from the tokenizer’s output Ω_0 , which is fixed by the example. Compositions requiring objects that Ω_0 does not supply are eliminated by the availability filter, which is exact at this layer rather than heuristic. A candidate that survives is executable end to end.

The loss measured is output-side only. We do not currently score the residual between a candidate’s derived Ω_{in} and Ω_0 ; unsatisfiable candidates are filtered rather than penalised. A per-layer input-residual term would give the search a graded signal for how far a composition is from closing, rather than a binary one, and we note it as an available extension that the present results do not use.

3.7 Training loop

Examples are processed one at a time: forward pass, loss, backward pass, with the surviving top- k programs carried to the next example. Pruning is on *cumulative* score across all examples seen so far, not on the current example alone, so a program that suits early examples and fails later ones is displaced rather than retained.

Program search and primitive training alternate in phases. Primitives are frozen during search, making each phase’s search a stationary problem and cumulative scoring meaningful. Between phases, the backward signals—the per-step required ontologies and the contract violations localised to specific primitives—supply training targets, and the neural primitives are fine-tuned. After retraining, the surviving beam is rescored under the new primitives before the next phase begins; carrying old scores forward would rank programs by the performance of primitives that no longer exist, and retraining improves precisely the primitives that were failing.

Mean beam loss before and after each retraining phase gives the convergence curve, and provides a stopping rule: halt when a retraining phase no longer improves it.

The base model is never updated. What is learned is the program—the composition and its parameters—together with the weights of the small neural primitives it invokes. Capability therefore resides in an inspectable artifact and in a set of independently verifiable, independently retrainable components, rather than entangled in the weights of the model being served.

Hyperparameters. N (layers), k (beam width), d_{max} (composition depth), and other hyperparameters appear in Appendix A.

4 Experiments

4.1 Benchmarks

We evaluate on two benchmarks chosen for distinct verification characters rather than for topical breadth. LiveCodeBench v6 (03/25-04/25) verifies by executing generated code against tests. τ^2 -Bench verifies constraint satisfaction over a multi-turn dialogue against a stated policy. A method whose advantage depended on one kind of checkable output would not transfer across both the tasks.

4.2 Models

The base models for all primary results are Gemma 3 12B, Muse Glimmer 30B, GPT-OSS-20B, and Nemotron-3-Nano 30B. Neural primitives are fine-tuned from two 3B models, three 8B models, and one 27B model. All primitives are resident at inference; we report parameters in the convention used for mixture-of-experts models, since the situation is analogous: $\approx 57\text{B}$ total, 3–27B active per forward pass.

To show comparison with other post-training approaches, three additional base models are evaluated under RL. Mid-tier and frontier models appear under vanilla prompting and under a token-matched agentic harness; we do not fine-tune them.

4.3 Primitive fine-tuning

Each neural primitive is a separately fine-tuned small language model, trained on a single operation. This subsection reports how those models are produced; the program that composes them is learned separately (§4.7).

Corpora. Training examples pair a task-specific system prompt with a structured JSON target, and are generated synthetically. Each example is constructed for the primitive’s operation rather than for any benchmark’s task: the target is the structured object the primitive must produce, and the exact-match criterion used in validation requires the produced object to match it. Table 1 reports the corpus size per primitive with a 90/10 train/validation split throughout.

Task-agnosticism and provenance. Primitives are trained on their operation, not on any benchmark’s task. A primitive that orders tasks by dependency is trained to order tasks by dependency, whatever the domain. This is what makes one library serve several tasks, and it is also what forecloses the reading that our results are distillation: distillation transfers task-specific behaviour, and no primitive is trained on a benchmark’s task. Because the corpora are entirely synthetic, we state the generating model explicitly, screen all corpora against the two evaluation sets, and report the overlap found (§4.1).

Base models. Primitives are not fine-tuned from a single base. Each is initialised from the smallest model that suffices for its operation, which is the practical form of the claim that reliability is inversely related to task scope: a primitive doing one narrow job does not need the capacity of a model doing many. ELABORATE and DECOMPOSE use Granite 4.1 3B IBM Research (2026), DENOTE uses Qwen 3.6 27B Qwen Team (2026), and the remainder use Granite 4.1 8B IBM Research (2026).

DENOTE is the exception that explains the rule. It must produce a term that parses and type-checks in a formal target language, which is the one operation in the library whose output is judged by an external checker rather than by structural agreement with a target, and it is allocated the largest base accordingly. ELABORATE and DECOMPOSE, despite carrying the two largest corpora, run on the smallest base: their difficulty is in coverage and grounding, which the data supplies, rather than in capability.

Procedure. Primitives are fine-tuned for 4 epochs at learning rate 1×10^{-5} . Each primitive is validated independently against its own output contract before integration, using both structure and value based constraints: validation for GET-ORDER, for instance, checks that the produced ordering respects all declared dependencies and is a permutation of its input. Primitives failing a reliability threshold on validation are retrained with additional data before deployment.

Table 1: Fine-tuning corpora and base models. Corpora are synthetic throughout, split 90/10.

Primitive	Base model	Total	Train	Val
DECOMPOSE	Granite 4.1 3B	6,910	6,218	692
CHECK-PREREQUISITES	Granite 4.1 8B	4,867	4,376	491
ELABORATE	Granite 4.1 3B	2,000	1,800	200
GET-ORDER	Granite 4.1 8B	2,000	1,800	200
MATCH-FUNC	Granite 4.1 8B	1,600	1,440	160
DENOTE	Qwen 3.6 27B	1,125	1,011	114
Total		18,502	16,645	1,857

Retraining across phases. The figures above are for the first phase. In later phases, training targets come from the backward pass: per-step required ontologies and contract violations localised to specific primitives (§3.7). We report per-primitive accuracy at each phase boundary and the accumulated error attributed to each primitive across the dataset, which is what determines the retraining targets for the next phase.

4.4 Baselines

Baselines sweep the design space rather than enumerate available systems. Each answers a distinct objection.

- **Vanilla prompting** — the floor. Function signatures, turn history, and the request in context.
- **Agentic harness (RLM), token-matched.** Answers whether the gain comes from scaffolding, and—because it is matched on inference tokens to our programs—whether it comes from test-time compute. We tune the harness’s iteration budget so the arm lands near our token count in expectation rather than truncating trajectories mid-run, and report which was done.
- **Post-training (RL).** Published checkpoints, described below.

4.5 Post-training baselines

We do not train supervised or reinforcement-learning baselines ourselves. Published checkpoints exist at both the data scale and the parameter scale relevant to our claim, and comparing against them removes any question of whether a baseline was competently trained: we did not choose their hyperparameters, their data, or when to stop. We make no claim relative to supervised fine-tuning and compare only against RL-post-trained models, which is the stronger target.

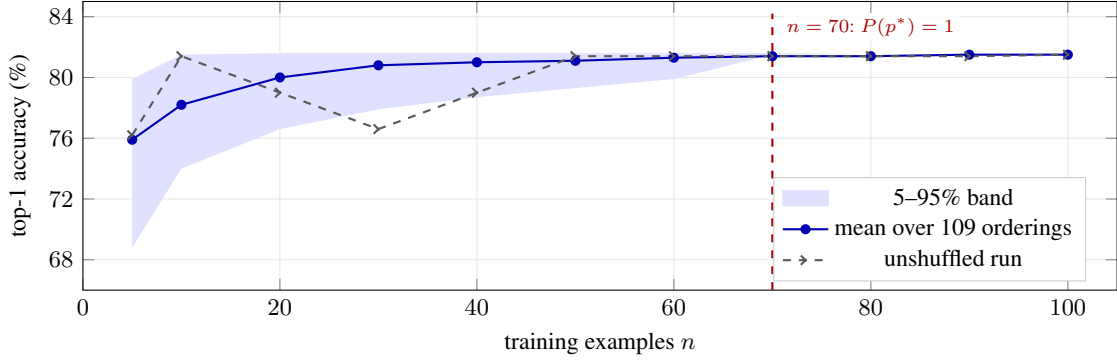


Figure 1: Top-1 accuracy of the leading indifference class against training-set prefix length. The unshuffled run reaches its final accuracy at $n = 10$ and does not move thereafter, but is leading with a program other than p^* until $n = 100$. The red line marks $n = 70$, where p^* lies in the leading class under every permutation.

Because no single checkpoint matches us on both axes, we report two kinds of comparison. They answer different objections and are not interchangeable.

Data-matched. Checkpoints post-trained on a corpus comparable to our 19,627 primitive fine-tuning examples, at a smaller parameter count than ours.

- **Competitive coding: DeepCoder-14B-Preview.** Luo et al. (2025) RL over 24K verifiable coding problems drawn from TACO Verified, PrimeIntellect SYNTHETIC-1, and LiveCodeBench 05/01/23–07/31/24, each verified against an official solution and required to carry at least five unit tests, then deduplicated and decontaminated against LiveCodeBench 08/01/24–02/01/25. Approximately 2.5 weeks on 32 H100s.
- **Tool calling: Nemotron-Cascade-2-30B-A3B.** Yang et al. (2026) 30B total, 3B active, post-trained from Nemotron-3-Nano-30B-A3B-Base with Cascade RL applied sequentially by domain alongside multi-domain on-policy distillation. Checkpoints and training data are both released, and it reports both LiveCodeBench and τ^2 -Bench, which is why we treat it as the primary post-training comparison.

Both are trained on more task-specific data than we use and both are smaller than our composed system, so the comparison is favourable to us on parameters and unfavourable on data.

Parameter-matched. A checkpoint at comparable parameter count, post-trained without a data-scale constraint.

- **INTELLECT-3 Team et al. (2025)**, 106B total with 12B active, post-trained from GLM-4.5-Air-Base through two SFT stages and large-scale RL on 512 H200s over two months. Its coding data includes 1.9M examples in the general reasoning SFT stage and an RL environment of 8.6K annotated programming problems verified by executing up to fifteen test cases each. Its tool-use data includes 800K examples in the general SFT stage and, in a dedicated agentic stage, 116K from Toucan (synthesised from real MCP environments), 10.3K from SWE-Swiss, and 38.4K from its environments mix; RL adds tool-using environments for deep research (2.2K samples) and software engineering.

Against our 19,627 examples this is roughly two orders of magnitude more task-specific data, so the arm is parameter-matched and deliberately not data-matched. On active parameters — 12B against our 3–27B — the match is closer than on total parameters, and we report both.

The claim we make is bounded by the weaker of the two comparisons: PLVR exceeds RL post-training at matched data, and at comparable parameters despite using a fraction of the task-specific data.

4.6 Convergence of program search

Program search is run over a prefix of the training set and the leading indifference class recorded at each checkpoint. Because a single training order is one draw from the space of orderings, we report the distribution over 109 permutations of the training set with the candidate pool held fixed: solid line is the mean over permutations, shaded region the 5–95% band. The single unshuffled run the search actually performed is overlaid as a dashed line. Exact figures for every checkpoint appear in Appendix B.

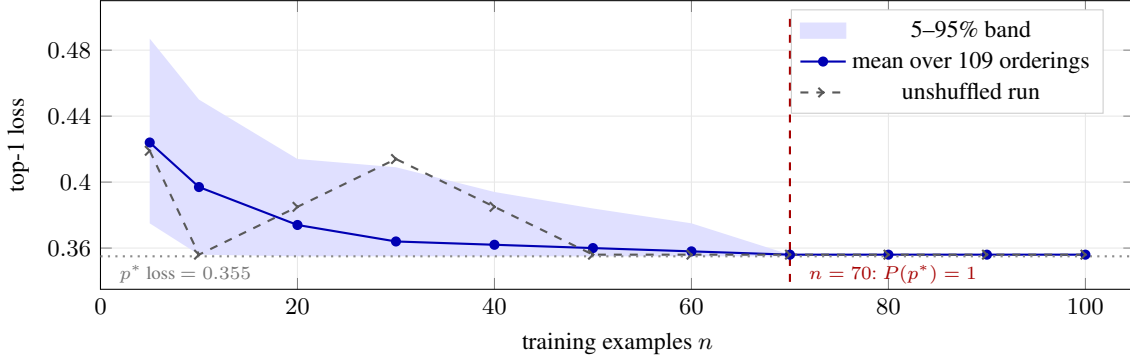


Figure 2: Top-1 loss of the leading indifference class against training-set prefix length. The band collapses onto the p^* loss floor at $n = 70$. The unshuffled run’s non-monotonicity—improving at $n = 10$, degrading through $n = 30$, recovering by $n = 50$ —is a property of that ordering and is absent from the mean.

4.7 Protocol

Runs and statistics. Every arm is run 3 times. For the arms we train or search — vanilla prompting, the token-matched harness, and PLVR — repetitions vary the inference and search seeds.

The post-training baselines are published checkpoints (§4.5), so there is no training seed for us to vary: their repetitions measure inference variance only, and we do not present their intervals as though they characterised training variance. This is the price of not training baselines ourselves. We pay it deliberately: a baseline whose hyperparameters, data, and stopping point we did not choose cannot be accused of having been under-trained, and that objection is the one most damaging to our claim. Where a checkpoint’s own authors report a benchmark we also evaluate, we give both their figure and ours, so any discrepancy in harness or decoding settings is visible rather than absorbed.

All arms share the same problem order and the same τ^2 user-simulator seeds, so comparisons are paired. Seeds are fixed a priori and reported.

Our effective sample size is problems, not seeds. We therefore report bootstrap confidence intervals over problems as the primary interval and give individual run numbers in Appendix C; a standard deviation estimated from three points is not informative and we do not present one as though it were.

Inference settings for baselines. Checkpoints are evaluated through official provider APIs where available, rather than self-hosted, to avoid performance loss from quantisation or other serving-side optimisations.

Compute accounting. A composed program spends more inference tokens than a single forward pass, and its primitives are separately fine-tuned from bases of three different sizes (§4.3). We report total parameters, active parameters, and inference tokens split into input and output, rather than the unit that flatters us. Active parameters vary with which primitive is executing, so we report the range and the trace-weighted mean. Input tokens dominate, since context is re-sent to each primitive; we report the ratio and state whether caching is used.

For the baselines we report the training scale their authors published — corpus size, hardware, and duration where given — rather than re-deriving it. Those figures are what make the data-matched and parameter-matched distinctions (§4.5) checkable rather than asserted, and a baseline whose training scale is undisclosed is not usable for either comparison, which is why our selection was constrained to checkpoints that report it.

We do not attempt FLOP-matched comparison. Making training FLOPs comparable across seven heterogeneous fine-tunes, a program search, and three externally trained checkpoints on schedules we did not control is not something we can do correctly, and a wrong FLOP figure is worse than none. We report the raw components instead — GPU-hours for primitive fine-tuning, candidate evaluations during program search, and each baseline’s published training scale — and leave the conversion to the reader.

We do not report latency. It is dominated by deployment optimisation rather than by method, and this is not a latency paper; token counts and memory footprint are reported instead.

Pre-registration. The random-sampling null (§5.5) was specified before it was run. Three choices were fixed in advance: the budget schedule, that budget is matched on programs drawn rather than wall-clock, and that the null retains

Table 2: Accuracy (%) by benchmark. PLVR rows wrap the reasoning layer around the base model directly above their group. Average is over the four columns; [†] marks an average over the columns available.

System	τ^2 -Bench			LCB	Avg
	Airline	Retail	Telecom		
<i>Base models, vanilla prompting</i>					
Gemma 12B	47.3	5.3	5.2	28.7	21.6
GPT-OSS-20B	40.0	21.1	23.1	48.8	33.2
Nemotron 3 Nano	46.7	33.9	28.1	55.9	41.2
Muse Glimmer 30B	69.3	43.3	65.2	67.9	61.4
<i>Post-training baselines (RL)</i>					
DeepCoder-14B	—	—	—	55.0	55.0 [†]
Nemotron-Cascade-2 30B-A3B	60.0	67.8	41.5	—	56.4 [†]
INTELLECT-3 106B-A12B	58.7	59.9	23.4	57.6	49.9
<i>Harness baseline</i>					
GPT-OSS-120B + RLM	21.2	34.6	54.8	81.0	47.9
<i>Mid-tier and frontier, vanilla prompting</i>					
Inkling 975B	59.3	45.9	74.9	35.4	53.9
GPT-5.6 Terra	66.0	40.4	51.7	85.0	60.8
Kimi K3	62.7	38.3	57.9	84.2	60.8
GPT-5.6 Sol	76.7	42.1	55.0	80.8	63.7
DeepSeek-V4-Flash	83.3	77.2	86.3	52.5	74.8
Claude Opus 5	67.3	47.4	95.3	90.0	75.0
<i>PLVR (ours)</i>					
+ Gemma 12B	56.0	28.1	87.7	83.8	63.9
+ Nemotron 3 Nano	70.0	72.8	—	77.5	73.4 [†]
+ GPT-OSS-20B	70.0	67.5	91.2	85.0	78.4
+ Muse Glimmer 30B	82.0	57.0	98.2	85.0	80.6
+ Inkling 975B	70.0	79.8	100.0	82.5	83.1

type-directed propagation, $d_{\max}$, and availability filtering with the correct `layers_below`. The last is what determines whether the comparison isolates the scoring or confounds it with the substrate, and fixing it after seeing results would have made the null unfalsifiable.

5 Results

5.1 Benchmark results

Table 2 reports all systems on three τ^2 -Bench domains and LiveCodeBench v6 (Mar–Apr). Every figure is the mean of 3 valid runs; runs scoring exactly zero or failing to produce a score are treated as failed attempts and excluded, and per-run values with standard deviations appear in Appendix C. PLVR rows are the same reasoning layer wrapped around each base model, with no change to the base model’s weights.

Every base model improves, and the weakest improve most. Table 3 isolates the effect of the reasoning layer with the base model held fixed.

The improvement is present on every model and every benchmark measured, ranging from 13.8 to 80.7 points. It is largest where the base model is weakest—GPT-OSS-20B gains 48.5 points on average from a vanilla average of 30.0—which is consistent with the mechanism: contract checking and recovery add value in proportion to the number of step-level errors there are to catch, and weaker models make more of them. The same pattern appears within a model: Gemma 12B gains 8 points on Airline, where it already scores 47.3, and 80.7 on Telecom, where it scores 5.2.

Small models with PLVR exceed frontier models without it. PLVR on Gemma 12B averages 63.9, above Kimi K3 (60.8), GPT-5.6 Terra (60.8), and GPT-5.6 Sol (63.7). On GPT-OSS-20B it averages 78.4 and on Muse Glimmer 30B

Table 3: Gain from PLVR over the same base model, in accuracy points. [†] average over available columns.

Base model	τ^2 Airline	τ^2 Retail	τ^2 Telecom	LCB v6(Mar-Apr)	Avg
Gemma 12B	+8.0	+22.8	+80.7	+56.3	+42.0
GPT-OSS-20B	+36.0	+48.2	+68.4	+41.2	+48.5
Nemotron 3 Nano	+34.0	+44.7	—	+13.8	+30.8 [†]
Muse Glimmer 30B	+16.0	+21.0	+32.4	+21.3	+22.7
Inkling 975B	+16.0	+35.9	+21.1	+48.7	+30.4

80.6, both above Claude Opus 5 (75.0) and DeepSeek-V4-Flash (74.8), the strongest vanilla systems measured. The best configuration, PLVR on Inkling 975B, averages 83.1.

We state the comparison this way deliberately: these frontier figures are vanilla prompting, and a frontier model with a comparable harness is a different and fairer opponent. The RLM row is the harness comparison we do run—GPT-OSS-120B with a scaffold reaches 47.9, below every PLVR configuration and below three of the five vanilla frontier systems.

Against RL post-training. All three post-training baselines fall below every PLVR configuration on their common columns. On the three τ^2 domains, Nemotron-Cascade-2 averages 56.4 and INTELLECT-3 47.3, against 57.3 for PLVR on Gemma 12B and 76.2 on GPT-OSS-20B. On LiveCodeBench, DeepCoder-14B reaches 55.0 and INTELLECT-3 57.6, against 83.8 for PLVR on Gemma 12B.

The data-matched reading is DeepCoder and Nemotron-Cascade-2: both trained on corpora comparable to or larger than our 19,627 primitive examples, both smaller than our composed system. The parameter-matched reading is INTELLECT-3 at 106B total and 12B active, trained on roughly two orders of magnitude more task-specific data. PLVR on a 12B base exceeds both.

Taking Muse Glimmer 30B as the reference configuration, the average gap over the three RL baselines on their shared columns is 27.8 points—+30.7 against INTELLECT-3, +30.0 against DeepCoder-14B on LiveCodeBench, and +22.6 against Nemotron-Cascade-2 on the τ^2 domains—and 13.6 points over the mean of the five frontier systems under vanilla prompting.

Where the result is weakest. Two caveats belong here rather than in a footnote. τ^2 Retail is the benchmark where PLVR leads least comfortably: our best configuration reaches 79.8 against DeepSeek-V4-Flash at 77.2, a 2.6-point margin that three runs cannot resolve confidently. And the Telecom column carries the largest gains in the table—80.7 points on Gemma 12B—from vanilla scores near zero. A base model scoring 5.2% is failing structurally rather than marginally, so the gain there measures the recovery of a collapsed baseline more than an improvement over a working one. We report the per-benchmark figures alongside the average for this reason and would not rest the claim on Telecom alone.

5.2 Preliminaries: the indifference class

Ranking programs by a strict order treats a 0.001 difference in loss as decisive. Primitives are language-model calls, so re-running a program moves its score; a difference below that noise floor is not evidence. We therefore rank *classes* rather than individual programs.

The **indifference class** $I_n(p)$ is the set of programs whose scores on the first n examples lie within tolerance of the leader’s:

$$I_n = \{p : \hat{R}^n(p) \leq \min_q \hat{R}^n(q) + \tau\} \quad (8)$$

with τ set at ± 0.10 RMS loss, ± 1.5 accuracy points, and ± 1 type error.

This is not an equivalence class. Its members are genuinely different programs—we verified two members differing on 17 of 60 examples—that are merely indistinguishable *at this sample size*. The relation describes the state of our evidence, not a property of the programs. It is indexed by n and narrows as evidence accumulates, which is why class size varies between 1 and 3 in the tables below. “Top- k classes” means pooling the members of the k leading classes, so shortlist size reflects how many programs are genuinely tied rather than being forced to a fixed k .

Pool and behaviours. We distinguish two counts used throughout. The retained **pool** is 213 programs exhibiting 23 distinct behaviours, and is the object analysed in Sections 5.5 and 5.6. Of these, **13 behaviours were discovered during the tracked training run** analysed in Section 5.4; the remainder entered the pool in other phases. Where we report birth points and changes to the leading class, the figures refer to those 13.

5.3 Convergence and identification

Let p^* denote the program with the best score measured on all 109 examples: 81.6% accuracy at 0.355 loss. Convergence means that the criterion, ranking on a *prefix* of n examples, eventually places p^* in its leading class.

Figures 1 and 2 (§4.6) report both quantities against n ; exact per-checkpoint values, including $P(p^* \in \text{top-1})$, are in Appendix B. Averaged over permutations, identification is reliable from $n = 70$: the leading class contains p^* in every ordering, and the 5–95% band collapses to a point. **Program search converges on this task within seventy examples**, and we report that figure rather than the length of any single run.

Accuracy converging is not identification succeeding. The two are routinely conflated, and separating them is the reason we report $P(p^*)$ rather than accuracy alone. In the unshuffled run, top-1 accuracy reads 81.4% from $n = 10$ onward and never moves—but the leading class does not contain p^* until $n = 100$. Throughout that range the criterion was leading with a *different* program of nearly identical score. An accuracy curve would have shown convergence at $n = 10$; identification had not occurred.

The two figures therefore measure different things and both are reported: 70 examples for identification averaged over orderings, and 100 for the unshuffled run. Averaged over 109 permutations that ordering is revealed as unlucky rather than typical, which is why the shuffled distribution is the primary result and the unshuffled run appears only as an illustration. Reporting one training order would have overstated convergence speed on accuracy and understated it on identification.

A non-monotonicity worth noting. $P(p^*)$ dips from 0.73 at $n = 5$ to 0.62 at $n = 10$ before climbing. At very small n many programs tie, so a wide leading class contains p^* by *inclusion* rather than by discrimination. As evidence narrows the class, p^* is briefly excluded before being placed back on merit. Early membership and late membership do not mean the same thing.

5.4 Discovery continues after best-in-pool plateaus

We initially expected that once best-in-pool accuracy saturates, the search has stopped contributing and subsequent changes are re-ranking of an effectively frozen candidate set. This is false, and the distinction matters for how many training examples are worth collecting.

The 13 behaviours discovered during this run are first seen at $n = 1, 1, 1, 5, 5, 7, 12, 12, 15, 33, 47, 47, 57$. Discovery continues to the end of training. Of the 15 changes to the leading indifference class, 4 are newly discovered programs *entering* it—at $n = 5, 7, 12, 33$ —and 11 are re-rankings of programs already in the pool. Roughly 27% discovery, 73% re-sorting.

Nothing better is found after $n = 5$ and nothing competitive is found after $n = 5$ are different claims, and only the first holds. A program first discovered at example 33 entered the leading class immediately on discovery.

This also qualifies the training-curve analogy. The leading class improves non-monotonically—15 accuracy points gained across 6 local decreases—and that is not purely estimator noise over a fixed candidate set. The candidate set grows underneath the ranking, the way an SGD iterate moves under a changing gradient estimate. Both mechanisms are active, and a plot of class quality should distinguish them rather than attributing the movement to one.

5.5 Ablation: removing the loss-guided search

This ablation removes the loss-guided component of the search and retains everything else, to establish whether the derivation contributes beyond what the type system and closure already provide. The null hypothesis it tests: at equal budget, uniform sampling from the *same* admissible space finds programs no better than loss-guided layer-wise search.

Construction. The ablated arm receives every constraint the full search has, minus the scoring. Ω_{req} propagates backward identically; each key is filled by a uniform draw from the type-admissible set at $d_{\text{max}} = 3$; availability filtering applies with the correct `layers_below`, so the final layer must close against Ω_0 . Anything the type system and closure provide, the null provides too—otherwise the comparison would credit the scoring with what the types already do. Both arms are scored on the same 109 examples under identical execution. We refer to the ablated arm below as the *null*.

Table 4: Expected best-found at equal N , where N is the number of programs drawn from each pool. Mean over 200 draw orders, so neither arm benefits from a fortunate ordering.

N	search pool	random pool	gap
5	77.6%	37.4%	+40.2
10	79.9%	44.3%	+35.5
20	81.2%	51.8%	+29.4
40	81.6%	62.6%	+19.0
60	81.6%	73.9%	+7.6

Table 5: Pool quality distributions. Quartiles are reported by value: the top quartile is the accuracy exceeded by the best 25% of programs in the pool, the bottom quartile the accuracy exceeded by the best 75%.

	n	best	top quartile	median	bottom quartile	$\geq 70\%$
search	213	81.6%	76.1%	65.6%	40.2%	96/213 (45%)
random	64	76.6%	35.2%	17.5%	0.0%	1/64 (2%)

The search pool reaches its optimum by $N = 40$. The null is at 62.6% there, and closes to within 7.6 points only at $N = 60$ —by which point it has drawn 60 of the 64 programs in its pool and is essentially exhausting its good tail.

The distributional result is the stronger one. 45% of the search’s pool clears 70% accuracy; 2% of the null’s does. The null’s bottom quartile is zero—a quarter of uniform draws produce programs that solve nothing at all. A best-of- N gap can be attributed to a fortunate draw; a median of 65.6% against 17.5% cannot. The guided search is not finding the same programs faster, it is populating a different region of the space.

5.6 Transfer to unseen examples

The transfer study uses a separate pool drawn from BFCL (Patil et al., 2025) and LiveCodeBench rather than τ^2 -Bench. The program selected on 60 examples was re-scored on a 109-example pool of which 73 were never seen during selection (42 BFCL, 31 LiveCodeBench). The pool, ranking, and winner were fixed before those examples were scored; nothing was tuned on them. Scoring is identical—real execution from Ω_0 , full test suites, 15 s timeout.

The selected program is

```
{Scratch ← INIT-SCRATCHPAD, Text ← ELABORATE, [Text] ← DECOMPOSE}
{[Code] ← WRAP(DENOTE), [ToolCall] ← FOREACH@0(MATCH)}
```

Accuracy falls 4.7 points from 82.3% in-sample to 77.6% held-out. The substantive question is whether that drop is selection bias—the program having been fitted to its selection set—or a difficulty difference between the sets. It is the latter, and the pool provides the control.

Every program in the pool drops 7–8 points on the held-out set, so the winner’s drop tracks the pool’s rather than exceeding it. The cause is visible in the problem families: the in-sample LiveCodeBench examples are entirely AtCoder Beginner Contest problems (abc374–378), while the held-out set adds abc379–386 together with arc184, arc185, arc186, and arc188—Regular Contest problems, a harder tier.

Tool-calling transfer is better than the pool’s. On BFCL the selected program goes 74.7% $\rightarrow$ 74.6%, a drop of 0.1 points, where the pool average drops 8.3. Selection on 60 examples returned a program markedly more robust on unseen tool tasks than the average candidate—the opposite of the overfitting one would expect from selecting on a small set.

6 Discussion

6.1 Types make candidates well-formed; the beam makes search finite

We initially believed that type-directed enumeration is what makes the search feasible—that unification and availability constrain the candidate set enough to score each layer exhaustively. Measuring the space shows this is wrong, and by a wide margin.

Table 6: In-sample and held-out performance of the selected program.

set	n	type err	RMS loss	accuracy	solved
selection set (in-sample)	60	2	0.352	82.3%	40/60
BFCL	30	2	0.412	74.7%	14/30
LCB	30	0	0.279	90.0%	26/30
held-out (unseen)	73	2	0.389	77.6%	43/73
BFCL	42	2	0.381	74.6%	19/42
LCB	31	0	0.399	81.7%	24/31
full	109	3	0.356	81.4%	71/109

Table 7: Pool-wide accuracy drop, measured across all 23 distinct behaviours in the retained pool rather than the winner alone.

benchmark	in-sample	held-out	pool-wide drop
LCB	63.9%	56.9%	−7.0
BFCL	58.5%	50.2%	−8.3

A single output layer covering $\Omega_{\text{out}} = \{[\text{ToolCall}], [\text{Code}]\}$ admits $533 \times 248 = 132,184$ plans. Each demands 5–7 input types (median 6), each independently satisfiable in 10^2 – 10^3 ways, so serving one layer-4 plan requires a median of 2.7×10^9 layer-3 plans. Compounding over four layers gives on the order of 10^{27} type-correct four-layer programs. The search scored roughly 14,400 candidates and retained 213—about 10^{-23} of the space.

10^{27} well-typed programs is not a tractable space by any construction. What makes the search finite is the **layer-wise beam**: eight partial programs survive each depth, so the branching factor is beam width times `max_plans` per layer rather than the full product. Types make each layer’s candidates *well-formed*; the beam makes the search *finite*. These are separate jobs and the first should not be credited with the second.

Availability prunes only at the bottom. With layers still to build, a required type counts as reachable if anything in the library could produce it, so nothing is cut. Only at layer 1—where `layers_below` is zero and the program must close against Ω_0 —does the filter bite, and there it is severe: $25\times$ on `[ToolCall]`, $19\times$ on `Scratch`. Closure, not typing, is the binding structural constraint, and it applies last.

Width dominates depth. Increasing d_{max} grows candidates per key polynomially; increasing *width* multiplies keys. A layer demanding six types multiplies the space by roughly 10^9 regardless of d_{max} . This is why `max_plans` dominated the run’s cost—raising it from 60 to 200 made the first phase unfinishable—while $d_{\text{max}} = 3$ remained affordable throughout. A practitioner budgeting this search should spend on depth before width.

6.2 Two readings the tables invite, and which one survives

Best-of- N understates the null result. Table 4 shows the gap narrowing from 40.2 points at $N = 5$ to 7.6 at $N = 60$, and read on its own that trajectory suggests the guided search buys efficiency which enough random draws would eventually recover. It does not, and the convergence is an artefact of the comparison rather than a property of the method. The null’s pool holds 64 programs; at $N = 60$ it has drawn 60 of them and is exhausting its good tail. Any best-of- N curve over a finite pool converges to that pool’s maximum, so the closing gap measures pool size, not search quality.

Table 5 is not subject to that reading. A median of 65.6% against 17.5%, with 45% of the guided pool clearing 70% against 2% of the null’s, is a statement about where in the space each procedure lands rather than about how many draws it takes to find one good program. A fortunate draw can produce a best-of- N gap; it cannot move a median across 213 and 64 programs respectively. We therefore treat the distribution as the primary evidence and the best-of- N table as supporting, and we would draw the same conclusion from the distribution alone.

The transfer result says something about the loss, not the search. The most surprising number in Section 5.6 is not the 4.7-point headline drop but the 0.1-point BFCL drop against a pool average of 8.3. The selected program is not

Table 8: Type-correct compositions per required type at $d_{\max} = 3$ over the primitive library, before and after availability filtering at layer 1, where the program must close against Ω_0 .

required type	raw candidates	admissible at layer 1	reduction
[ToolCall]	533	21	25×
[Code]	248	63	4×
Scratch	534	28	19×
Text	134	134	none
[Text]	1,037	1,037	none

merely the best on the selection set; it is markedly more robust on unseen tool tasks than the average candidate in the pool it was selected from.

Selection on 60 examples might reasonably have been expected to return a program fitted to those examples, degrading at least as fast as the pool on unseen ones. It returned the opposite. Since every arm in this paper ranks candidates under the same loss, and only the proposal mechanism varies, this is evidence about the ranking criterion rather than about the derivation: type errors ordered strictly before value errors, with content alignment, appears to prefer programs whose correctness is structural rather than incidental to the examples at hand.

We flag this as an observation rather than a claim. We did not design an experiment to test it, a single selected program on one benchmark is thin evidence, and the LCB half of the same table shows the winner tracking the pool rather than beating it. Establishing that a lexicographic type-first loss systematically selects for robustness would require varying the loss with the search held fixed—the one ablation this paper does not run.

6.3 What the results support, and what they do not

The null result establishes that the loss-guided search populates a qualitatively different region of the program space than uniform sampling over the same admissible set: not a faster route to the same programs, but a different distribution. The convergence result establishes that identification of the best available program is reliable within seventy examples, and that this is distinct from—and slower than—accuracy appearing to plateau. The transfer result establishes that selection on 60 examples does not overfit, with the winner’s degradation tracking the pool’s and its tool-calling performance holding almost exactly.

Three limits are worth stating plainly. The held-out split is not i.i.d., so we report a difficulty effect rather than a generalisation gap. The loss function and the backward pass are held fixed across all arms and are not separately isolated; what we have shown is that target-directed derivation beats other proposal mechanisms *under this loss*, not that the loss design is independently responsible. And a program well-typed on the training distribution carries no static guarantee on branch paths never exercised—though when such a path produces an off-type ontology at deployment, the relevant contract fires and the trace identifies the failing primitive, so the failure is detected and localised rather than propagating silently.

7 Conclusion

We introduced PLVR, a post-training method that leaves the base model’s weights untouched and instead learns an explicit program composed from deterministic and neural primitives. Its mechanism, symbolic backpropagation, replaces the estimated credit assignment of policy-gradient methods with a derivation: given the ontology a layer must produce, the declared signatures of the primitives induce the ontology the layer below must supply, and typed contracts localise failure to the step that produced it. Where RLVR verifies a terminal outcome, the reward here is a per-step contract verdict, dense over the program’s structure. Empirically, the guided search populates a different region of the program space than uniform sampling over the identical admissible set—a median of 65.6% against 17.5%—rather than merely reaching the same programs sooner; identification of the best available program is reliable within seventy examples; and the selected program transfers to unseen examples without the degradation that selection on sixty examples would ordinarily produce. Two findings run against our own initial expectations and we report them as such: types make candidates well-formed but do not make the space tractable—the layer-wise beam does—and discovery continues long after best-in-pool accuracy has plateaued, so a saturating accuracy curve is not evidence that further examples are worthless. We release the symbolic backpropagation library together with a conformance checker, so that the method can be applied to any primitive set satisfying the interface. The claim we would most like tested by others is not a benchmark number but a property of substrates: that running a program- construction procedure over typed,

contract-verified primitives outperforms running the same procedure over raw model calls. That is checkable by anyone with a compositional pipeline and a way to verify its intermediate steps, and it does not require our code.

Acknowledgements

We thank Ram Shanmugam and Jay Vaghasiya for helping with comments on the earlier drafts and helping with experimental results.

Reproducibility Statement

The method is specified in §3 and the settings needed to reproduce our runs in Appendix A, with per-primitive corpora and base models in Table 1, per-checkpoint convergence values in Appendix B, and per-run baseline scores in Appendix C. Post-training baselines are published checkpoints, identified by their model-hub paths. The symbolic backpropagation library and conformance checker will be released following peer review.

References

- Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. Gega: Reflective prompt evolution can outperform reinforcement learning, 2026. URL <https://arxiv.org/abs/2507.19457>.
- Debangshu Banerjee, Changming Xu, Eugene Ie, Ming Zhang, Daiyi Peng, Chu-Cheng Lin, and Gagandeep Singh. Severa: Verified synthesis of self-evolving agents, 2026. URL <https://arxiv.org/abs/2603.25111>.
- Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. τ^2 -bench: Evaluating conversational agents in a dual-control environment, 2025. URL <https://arxiv.org/abs/2506.07982>.
- Ching-An Cheng, Allen Nie, and Adith Swaminathan. Trace is the next autodiff: Generative optimization with rich feedback, execution traces, and llms, 2024. URL <https://arxiv.org/abs/2406.16218>.
- Paul Christiano, Jan Leike, Tom B. Brown, Miljan Martic, Shane Legg, and Dario Amodei. Deep reinforcement learning from human preferences, 2023. URL <https://arxiv.org/abs/1706.03741>.
- Debargha Ganguly, Srinivasan Iyengar, Vipin Chaudhary, and Shivkumar Kalyanaraman. Proof of thought : Neurosymbolic program synthesis allows robust and interpretable reasoning, 2024. URL <https://arxiv.org/abs/2409.17270>.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Hanwei Xu, Honghui Ding, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jingchang Chen, Jingyang Yuan, Jinhao Tu, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaichao You, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingxu Zhou, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, S. S. Li, Shuang Zhou, Shaoqing Wu, Tao Yun, Tian Pei, Tianyu Sun, T. Wang, Wangding Zeng, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, W. L. Xiao, Wei An, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaotao Nie, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, X. Q. Li, Xiangyue Jin, Xiaojin Shen, Xiaosha Chen, Xiaowen Sun, Xiaoxiang Wang, Xinnan Song, Xinyi Zhou, Xianzu Wang, Xinxia Shan, Y. K. Li, Y. Q. Wang, Y. X. Wei, Yang Zhang, Yanhong Xu, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Yu, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yuan Ou, Yuduan Wang, Yue Gong, Yuheng Zou, Yujia He, Yunfan Xiong, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Y. X. Zhu, Yanping Huang, Yaohui Li, Yi Zheng, Yuchen Zhu, Yunxian Ma, Ying Tang, Yukun Zha, Yuting Yan, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhenda

- Xie, Zhengyan Zhang, Zhewen Hao, Zhicheng Ma, Zhigang Yan, Zhiyu Wu, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Zizheng Pan, Zhen Huang, Zhipeng Xu, Zhongyu Zhang, and Zhen Zhang. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638, 2025. ISSN 1476-4687. doi: 10.1038/s41586-025-09422-z. URL <http://dx.doi.org/10.1038/s41586-025-09422-z>.
- Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems, 2025. URL <https://arxiv.org/abs/2408.08435>.
- IBM Research. Introducing the IBM Granite 4.1 family of models. <https://research.ibm.com/blog/granite-4-1-ai-foundation-models>, April 2026. Dense decoder-only language models at 3B, 8B and 30B, base and instruct. Apache 2.0. Accessed 2026-08-27.
- Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. Livecodebench: Holistic and contamination free evaluation of large language models for code, 2024. URL <https://arxiv.org/abs/2403.07974>.
- Ryo Kamoi, Yusen Zhang, Nan Zhang, Sarkar Snigdha Sarathi Das, Ranran Haoran Zhang, Wenpeng Yin, and Rui Zhang. Efficient prm training data synthesis via formal verification, 2026. URL <https://arxiv.org/abs/2505.15960>.
- Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. Dspy: Compiling declarative language model calls into self-improving pipelines, 2023. URL <https://arxiv.org/abs/2310.03714>.
- Zhuofeng Li, Haoxiang Zhang, Seungju Han, Sheng Liu, Jianwen Xie, Yu Zhang, Yejin Choi, James Zou, and Pan Lu. In-the-flow agentic system optimization for effective planning and tool use, 2026. URL <https://arxiv.org/abs/2510.05592>.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step, 2023. URL <https://arxiv.org/abs/2305.20050>.
- Michael Luo, Sijun Tan, Roy Huang, Ameen Patel, Alpay Ariyak, Qingyang Wu, Xiaoxiang Shi, Rachel Xin, Colin Cai, Maurice Weber, Ce Zhang, Li Erran Li, Raluca Ada Popa, and Ion Stoica. DeepCoder: A fully open-source 14B coder at O3-mini level. <https://www.together.ai/blog/deepcoder>, April 2025. Agentica and Together AI. Accessed 2026-08-27.
- Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback, 2022. URL <https://arxiv.org/abs/2203.02155>.
- Liangming Pan, Alon Albalak, Xinyi Wang, and William Wang. Logic-LM: Empowering large language models with symbolic solvers for faithful logical reasoning. In Houda Bouamor, Juan Pino, and Kalika Bali (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2023*, pp. 3806–3824, Singapore, December 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.248. URL <https://aclanthology.org/2023.findings-emnlp.248/>.
- Shishir G Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 48371–48392. PMLR, 13–19 Jul 2025. URL <https://proceedings.mlr.press/v267/patil25a.html>.
- Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chenguang Zhu, and Michael Zeng. Automatic prompt optimization with “gradient descent” and beam search, 2023. URL <https://arxiv.org/abs/2305.03495>.
- Qwen Team. Qwen3.6-27B: Flagship-level coding in a 27B dense model, April 2026. URL <https://qwen.ai/blog?id=qwen3.6-27b>.

- Prime Intellect Team, Mika Senghaas, Fares Obeid, Sami Jaghouar, William Brown, Jack Min Ong, Daniel Auras, Matej Sirovatka, Jannik Straube, Andrew Baker, Sebastian Müller, Justus Mattern, Manveer Basra, Aiman Ismail, Dominik Scherm, Cooper Miller, Ameen Patel, Simon Kirsten, Mario Sieg, Christian Reetz, Kemal Erdem, Vincent Weisser, and Johannes Hagemann. Intellect-3: Technical report, 2025. URL <https://arxiv.org/abs/2512.16144>.
- Jonathan Uesato, Nate Kushman, Ramana Kumar, Francis Song, Noah Siegel, Lisa Wang, Antonia Creswell, Geoffrey Irving, and Irina Higgins. Solving math word problems with process- and outcome-based feedback, 2022. URL <https://arxiv.org/abs/2211.14275>.
- Peiyi Wang, Lei Li, Zhihong Shao, R. X. Xu, Damai Dai, Yifei Li, Deli Chen, Y. Wu, and Zhifang Sui. Math-shepherd: Verify and reinforce llms step-by-step without human annotations, 2024. URL <https://arxiv.org/abs/2312.08935>.
- Zhiheng Xi, Chenyang Liao, Guanyu Li, Yajie Yang, Wenxiang Chen, Zhihao Zhang, Binghai Wang, Senjie Jin, Yuhao Zhou, Jian Guan, Wei Wu, Tao Ji, Tao Gui, Qi Zhang, and Xuanjing Huang. Agentprm: Process reward models for llm agents via step-wise promise and progress, 2025. URL <https://arxiv.org/abs/2511.08325>.
- Zhuolin Yang, Zihan Liu, Yang Chen, Wenliang Dai, Boxin Wang, Sheng-Chieh Lin, Chankyu Lee, Yangyi Chen, Dongfu Jiang, Jiafan He, Renjie Pi, Grace Lam, Nayeon Lee, Alexander Bukharin, Mohammad Shoeybi, Bryan Catanzaro, and Wei Ping. Nemotron-cascade 2: Post-training llms with cascade rl and multi-domain on-policy distillation, 2026. URL <https://arxiv.org/abs/2603.19220>.
- Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Zhi Huang, Carlos Guestrin, and James Zou. Textgrad: Automatic “differentiation” via text, 2024. URL <https://arxiv.org/abs/2406.07496>.
- Alex L. Zhang, Tim Kraska, and Omar Khattab. Recursive language models, 2026. URL <https://arxiv.org/abs/2512.24601>.
- Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. Aflow: Automating agentic workflow generation, 2025. URL <https://arxiv.org/abs/2410.10762>.

A Hyperparameters

All settings below are for the final run over 109 examples. Search and evaluation use deliberately different scoring configurations, described in §A.2; the distinction matters for interpreting reported numbers and we state it explicitly rather than folding both into one table.

A.1 Program search

Parameter	Value	Role
layers (N)	4	program depth
d_max	3	composition depth within one layer
layer_k	8	partial programs surviving each depth (beam width)
per_key_k	8	candidates retained per key of Ω_{req}
max_plans	60	layer plans emitted, heap-ordered by summed rank
batch	3	examples the scoring function averages over
shards	4	parallel search shards, dealt <code>tasks[i:4]</code>
workers	4	concurrent candidate scorings per shard
Ω_{out}	<code>{[ToolCall], [Code]}</code>	union over the task distribution

Table 9: Backward search settings. `layer_k` is the parameter that makes the search finite (§6.1); `max_plans` dominated wall-clock cost, and raising it from 60 to 200 made the search phase unfinishable within our budget.

Ω_{out} is the union of output types over the task distribution rather than a per-example target, so one program serves both benchmarks: the loss is keyed by the target ontology, and an unrequested `[Code]` object on a tool-calling example is not scored (§3.5).

A.2 Scoring: search versus evaluation

Scoring during search is cheap by design; scoring for every reported number is not. Conflating the two would let a fast approximate scorer inflate results, so the configurations are separated and only the second is ever reported.

Parameter	During search	Reported evaluation
<code>assume</code>	True while <code>layers_below > 0</code>	False
<code>execution</code>	lookahead against library closure	real, from Ω_0
<code>tests</code>	3-test prefix (LCB)	full public suite
<code>timeout</code>	3 s per test	15 s per test
<code>limit_pool</code>	—	200

Table 10: Scoring configurations. `assume` permits a required type to be treated as reachable while layers remain to be built; at layer 1 it is disabled and the program must close against Ω_0 . Every number reported in §5 uses the right-hand column.

A.3 Loss and ranking

Parameter	Value
<code>alignment</code>	CONTENT (not KEY)
<code>Code policy</code>	TESTS: <code>val_err = 1 - pass rate</code>
<code>ToolCall policy</code>	PARTIAL
<code>inversion_weight</code>	0.5 (order penalty)
<code>score</code>	(<code>type_count</code> , <code>val_{L2}</code>), lexicographic
<code>ranking key</code>	(UNMET, <code>type_errors</code> , <code>val_{L2}</code> , <code>deviation</code> , <code>size</code> , <code>label</code>)

Table 11: Loss and ranking configuration. The score is a property of an output; the ranking key adds search-control terms that are properties of a candidate’s relation to the search state (§3.5).

Content alignment rather than key alignment is load-bearing: under key alignment a prediction containing every correct call displaced by one setup call receives the maximum loss on both axes, because `pred[k]` is only ever compared to `tgt[k]`.

`inversion_weight` scales the order penalty applied after content matching, so a correct set of calls in the wrong order is distinguished from a wrong set rather than being collapsed with it.

A.4 Indifference classes

These tolerances are used in the convergence analysis (§5.3) and nowhere in the search itself. The search ranks candidates strictly; classes are formed post hoc to avoid treating differences below measurement noise as evidence.

Tolerance	Value	Quantity
τ_{loss}	± 0.10	RMS loss
τ_{acc}	± 1.5	accuracy points
τ_{terr}	± 1	type errors

Table 12: Indifference-class tolerances, applied in analysis only.

B Convergence: exact figures

This appendix gives the per-checkpoint values summarised in §5.3, for both the leading indifference class (top-1) and the three leading classes pooled (top-3). Two tables are reported: the unshuffled run, which is the single training order the search actually saw, and the distribution over 109 permutations of that order with the candidate pool held fixed.

Throughout, $|\cdot|$ denotes the number of programs in the reported class or pooled classes, and p^* is the program with the best score measured on all 109 examples (81.6% accuracy, 0.355 loss).

B.1 Unshuffled run

n	top-1			$ \cdot $	top-3			p^* in top-1
	acc (%)	loss			acc (%)	loss		
5	76.2	0.419	3		67.5	0.497	4	no
10	81.4	0.356	1		78.0	0.402	3	no
20	79.0	0.385	2		72.4	0.459	4	no
30	76.6	0.414	1		76.2	0.419	3	no
40	79.0	0.385	2		72.4	0.459	4	no
50	81.4	0.356	1		76.2	0.419	3	no
60	81.4	0.356	1		76.2	0.419	3	no
70	81.4	0.356	1		76.2	0.419	3	no
80	81.4	0.356	1		76.2	0.419	3	no
90	81.4	0.356	1		76.2	0.419	3	no
100	81.5	0.356	2		77.5	0.403	4	yes

Table 13: The unshuffled run: the training order the search actually saw, with the candidate pool gated by discovery. Top-1 accuracy reaches its final value at $n = 10$ and does not move, but the leading class does not contain p^* until $n = 100$.

Two features of this table are discussed in §5.3 and repeated here for reference. The class size $|\cdot|$ oscillates between 1 and 3 rather than shrinking monotonically, because a class narrows as evidence accumulates but widens again when a newly discovered program enters it. And accuracy stabilising at $n = 10$ while p^* remains outside the leading class until $n = 100$ is the reason we report identification separately from accuracy.

B.2 Distribution over 109 permutations

n	top-1		$ \cdot $	top-3		$ \cdot $	$P(p^*)$
	acc (%)	loss		acc (%)	loss		
5	75.9 [68.8, 79.9]	0.424 [.375, .487]		70.8 [58.4, 77.4]	0.477 [.412, .589]		0.73
10	78.2 [74.0, 81.5]	0.397 [.356, .450]		75.3 [70.8, 78.2]	0.432 [.403, .479]		0.62
20	80.0 [76.6, 81.6]	0.374 [.355, .414]		77.2 [74.3, 79.9]	0.410 [.375, .438]		0.72
30	80.8 [77.9, 81.6]	0.364 [.355, .409]		77.6 [74.3, 79.9]	0.405 [.375, .438]		0.84
40	81.0 [78.7, 81.6]	0.362 [.355, .394]		77.6 [74.3, 79.9]	0.404 [.375, .438]		0.85
50	81.1 [79.3, 81.6]	0.360 [.355, .384]		77.7 [74.9, 79.9]	0.403 [.375, .438]		0.88
60	81.3 [79.9, 81.6]	0.358 [.355, .375]		77.5 [74.3, 79.9]	0.404 [.375, .438]		0.94
70	81.4 [81.5, 81.6]	0.356 [.355, .356]		77.6 [76.9, 79.9]	0.403 [.375, .418]		1.00
80	81.4 [81.5, 81.5]	0.356 [.356, .356]		77.4 [77.4, 77.5]	0.404 [.403, .407]		1.00
90	81.5 [81.5, 81.5]	0.356 [.356, .356]		77.5 [77.5, 77.5]	0.403 [.403, .403]		1.00
100	81.5 [81.5, 81.5]	0.356 [.356, .356]		77.5 [77.5, 77.5]	0.403 [.403, .403]		1.00

Table 14: Mean with 5–95% band over 109 permutations of the training order, pool fixed. $P(p^*)$ is the probability that the leading class contains p^* .

B.3 Top-1 against top-3

B.4 Reading the top-3 curve

Top-3 tracks top-1 at a stable 3–4 point deficit in accuracy and a 0.047 deficit in loss from $n = 20$ onward. This is the expected direction—pooling three classes admits programs the leading class excludes—and its stability is the informative part: the leading class is genuinely leading rather than an arbitrary cut, and widening the shortlist buys coverage at a predictable and modest cost in quality.

The top-3 band is wider than top-1’s at every n and does not fully collapse, because the third class remains contested after the first has stabilised. A practitioner carrying a shortlist forward rather than a single program should expect the shortlist’s composition to keep changing after its leader has settled.

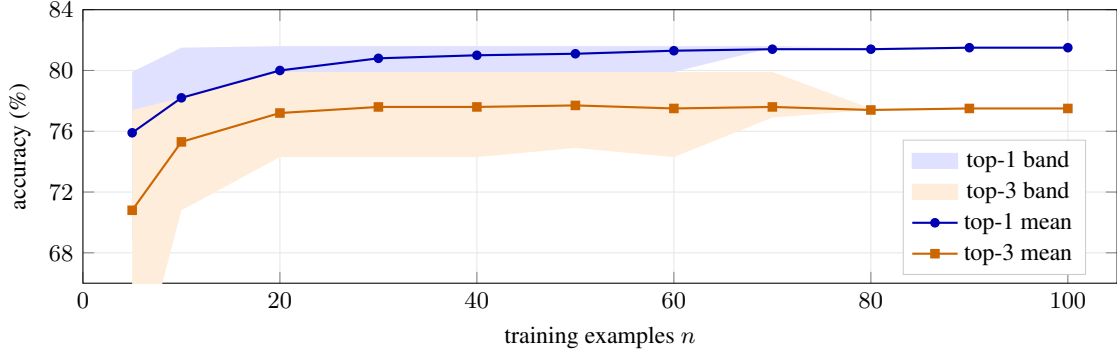


Figure 3: Accuracy of the leading class against the three leading classes pooled, over 109 permutations. Top-3 sits 3–4 points below top-1 from $n = 20$ onward and converges to a stable deficit rather than closing.

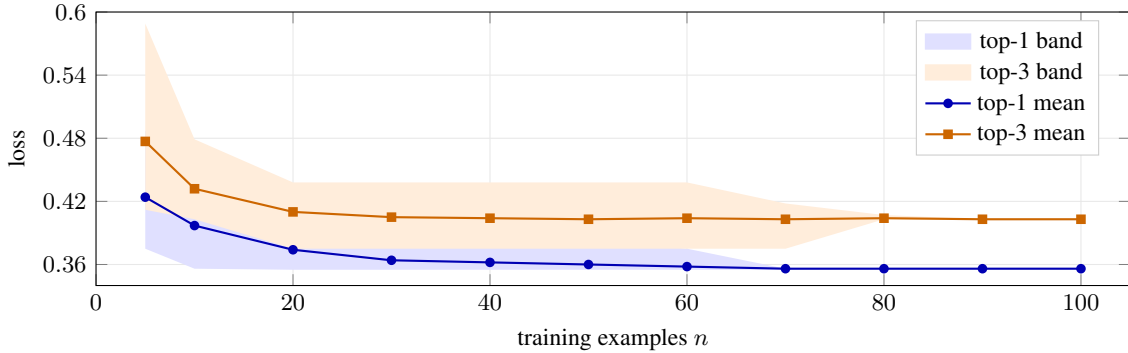


Figure 4: Loss of the leading class against the three leading classes pooled, over 109 permutations. Both converge; the top-3 floor at 0.403 is above the top-1 floor at 0.356 by construction, since pooling admits programs the leading class excludes.

Note. At $n = 70$ and $n = 80$ the reported top-1 mean accuracy (81.4) falls below the lower bound of its own 5–95% band (81.5). This is a rounding artefact of the source figures and not a property of the distribution; the underlying values differ by less than 0.1 points.

C Post-training baselines: per-run scores

This appendix gives every valid run behind the post-training rows of Table 2. Runs scoring exactly zero, or terminating without producing a score, are treated as failed attempts and excluded from the score list, mean, and standard deviation; the count of valid runs is reported per cell. Standard deviations are sample deviations ($n - 1$) over three points and are reported for completeness rather than as interval estimates—three points do not support one, which is why bootstrap confidence intervals over problems are the primary interval in §5.

All three are published checkpoints evaluated by us, not models we trained (§4.5). Repetitions therefore vary inference conditions only; there is no training seed for us to vary, and these deviations do not characterise training variance.

Coverage. Two cells in Table 2 are empty and the reason differs. DeepCoder-14B-Preview is post-trained for competitive programming and we do not evaluate it on the τ^2 domains, since a coding checkpoint scoring poorly on tool-calling dialogue would not be an informative comparison. Averages marked [†] in Table 2 are taken over available columns only, and we do not impute missing cells.

C.1 Intervals over problems

Standard deviations over three runs describe run-to-run variation, not the uncertainty that matters for a comparison. The dominant source of uncertainty is the finite problem set: τ^2 Airline has 50 tasks, Retail and Telecom 114 each, and

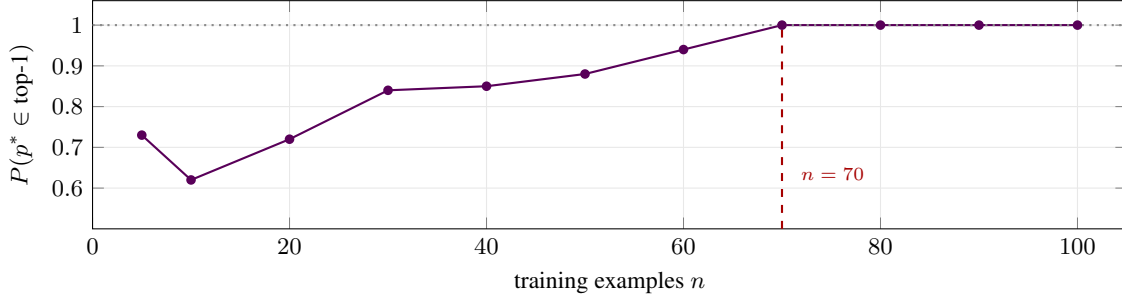


Figure 5: Probability that the leading class contains p^* , over 109 permutations. The dip at $n = 10$ occurs because a wide class at small n contains p^* by inclusion rather than discrimination; as the class narrows, p^* is briefly excluded before being placed back on merit.

Model	Benchmark	Mean	$\pm$ SD	n	Runs, oldest $\rightarrow$ newest (%)
<i>INTELLECT-3</i> PrimeIntellect/INTELLECT-3					
	τ^2 Airline	58.7	5.0	3	58.0, 54.0, 64.0
	τ^2 Retail	59.9	2.7	3	62.3, 57.0, 60.5
	τ^2 Telecom	23.4	2.2	3	21.1, 23.7, 25.4
<i>DeepCoder-14B-Preview</i> agentica-org/DeepCoder-14B-Preview					
	LCB v6	55.0	2.5	3	55.0, 52.5, 57.5
<i>Nemotron-Cascade-2-30B-A3B</i> nvidia/Nemotron-Cascade-2-30B-A3B					
	τ^2 Airline	60.0	3.5	3	56.0, 62.0, 62.0
	τ^2 Retail	67.8	2.5	3	69.2, 69.2, 64.9
	τ^2 Telecom	41.5	4.1	3	36.8, 43.8, 43.8

Table 15: Per-run scores for the post-training baselines. Cells left blank in Table 2 are omitted here rather than reported as zero: DeepCoder-14B is a coding checkpoint and we do not evaluate it on τ^2 -Bench

LiveCodeBench v6 (Mar–Apr) 80. We therefore report intervals over problems as the primary interval.

These intervals are wide, and τ^2 Airline is the reason. At 50 tasks, a 95% interval spans roughly 26 points, so no comparison on Airline with a margin below about 13 points is resolvable from marginal intervals. Several comparisons in Table 2 fall inside that range: PLVR on Gemma 12B reaches 56.0 on Airline against INTELLECT-3’s 58.7 and Nemotron-Cascade-2’s 60.0, and all three intervals overlap substantially. We do not claim a difference on that column.

Marginal intervals are conservative for these comparisons. Every system is evaluated on the same problems, so the quantity of interest is the paired difference rather than two independent proportions. A paired bootstrap over per-problem outcomes—resampling problems and recomputing the difference within each resample—removes the between-problem variance common to both arms and yields substantially tighter intervals than the marginals above. Overlapping marginal intervals therefore do not establish that a difference is absent. We report paired intervals for the comparisons in §5 and the marginals here for reference.

Spread. The largest deviation in this table is 5.0 points (INTELLECT-3 on τ^2 Airline, spanning 54.0 to 64.0); the smallest is 1.5. Two cells are worth noting for anyone comparing against these numbers. INTELLECT-3’s Airline spread of ten points across three runs is wide relative to its own margin over the other baselines on that domain, so its Airline mean should be read with more caution than the table’s other cells. Nemotron-Cascade-2 produced identical scores on two of three runs for both Retail (69.2, twice) and Telecom (43.8, twice), which is consistent with a largely deterministic decode at its documented default settings rather than with low-variance sampling.

Model	Benchmark	Mean	n_{prob}	Wilson 95%	Bootstrap 95%
INTELLECT-3	τ^2 Airline	58.7	50	[44.9, 71.2]	[44.0, 72.0]
	τ^2 Retail	59.9	114	[50.8, 68.5]	[50.9, 68.4]
	τ^2 Telecom	23.4	114	[16.6, 32.0]	[15.8, 31.6]
DeepCoder-14B	LCB v6	55.0	80	[44.1, 65.4]	[43.8, 66.2]
Nemotron-Cascade-2	τ^2 Airline	60.0	50	[46.2, 72.4]	[46.0, 74.0]
	τ^2 Retail	67.8	114	[58.7, 75.6]	[58.8, 76.3]
	τ^2 Telecom	41.5	114	[32.8, 50.6]	[32.5, 50.0]

Table 16: Marginal 95% intervals over problems. Wilson intervals are computed from the pooled success rate; bootstrap intervals are 20,000 resamples of the problem set. The two agree to within a point in every cell, as expected for a binomial proportion, and we report both so that the bootstrap figure can be checked against a closed form.

Reproducing these intervals. Both interval types above are computed from aggregate scores by reconstructing a success/failure vector with the observed success count, which is exact for a binomial proportion but does not use per-problem identities. The paired intervals reported in §5 use the actual per-problem outcome vectors; the resampling procedure and seed are given with the released evaluation scripts.